高精度加法：从数楼梯出发

主线：发现存不下 → 数位数组 → 逐位相加 → 处理边界 → 接回递推

学习目标：弄懂原理，独立完成一次大数加法，再把工具用到楼梯题中。可先读第 1～4 节，再读第 5～8 节并写代码。

---

01 / 先找核心问题

公式对了，为什么结果还会错?

这一节只弄清：需要升级的是“数的保存和相加”，不是重写楼梯思路。

楼梯每次可以上 1 阶，也可以上 2 阶。到达第 i 阶，最后一步只可能从第 i−1 阶或第 i−2 阶走过来。两类走法不会重复，合在一起就是总数。

第 i 阶的走法数
= 前一阶的走法数 + 前两阶的走法数

你原来的 a[1] = 1、a[2] = 2 和下面的关系都符合题意：

```
a[i] = a[i − 1] + a[i − 2];
```

真正的障碍：结果越来越大，一个 long long 放不下。
在常见的 64 位 long long 环境中，最大正整数是 9223372036854775807。这道题到第 92 阶时，走法数就超过它了，而题目允许算到第 5000 阶。
有符号整数溢出后，结果不再可靠；不能指望它一定变成某个负数。

格子多 ≠ 每格能装很大的数

long long a[5005] 表示有 5005 个格子，**每个格子**仍受 long long 的范围限制。把数组加长，不能让某一个元素突破范围。

我们需要的新工具

用**多个格子一起表示一个大整数**，再自己编写数位运算。这种方法叫**高精度运算**。本课只学非负整数的加法。

“高精度”在这里的意思是：超过普通整数类型的范围，仍能保存每一位，得到精确的整数结果。它仍受数组容量和内存限制，不是无限大。

> **按需查阅 · 为什么不改用 double?**
> double 能表示更大数量级的数，但有效数字有限。走法总数需要每一位都准确，近似值不符合题目要求。这里需要“大整数”，不是“小数”。

接下来要回答：一个数拆开以后，怎样保存才能方便相加?

一个大整数，用一整个数组保存

约定好“每格代表什么”，程序才知道怎样使用这些数字。

先拿 **144** 练习。它由 4 个一、4 个十和 1 个百组成。每个格子只放一个十进制数字，范围是 **0～9**。

x[0]	x[1]	x[2]
4	**4**	**1**
个位	十位	百位

数组下标：0 → 1 → 2　|　数位：个位 → 十位 → 百位

它表示的数是 4 × 1 + 4 × 10 + 1 × 100 = 144。数组没有把数改成 441，只是换了保存顺序。

为什么个位放在下标 0?

竖式加法从个位开始，进位交给下一位。把个位放在下标 0 后，程序就可以按 0、1、2…… 的顺序计算，进位从 j 交给 j+1。这是一种方便的约定，不是 C++ 强制的规则。

存的时候：低位在前。读成平常的数字时：高位在前。
计算从下标 0 往后走；输出从最高有效位往下标 0 走。两个方向不同，各有用途。

还要记一个“长度”

```
int x[10] = {4, 4, 1}; // 表示 144，其余元素初始化为 0
int lenX = 3;          // 这个数实际用了 3 位
```

名字	它负责什么	这里是多少
容量	数组总共准备了多少格	10 格
lenX	这个数实际用了多少位	3 位
最高有效位的下标	输出应该从哪里开始	lenX − 1，即 2

容量和长度是两件事。准备了 10 格，不代表每次都要输出 10 位。单独表示数字 **0** 时，我们约定长度为 1、第一格是 0。

完整小任务：设计一个数的存法

在纸上画出 **5070** 的数组表示，同时标出每格的下标、数位和有效长度。再沿输出方向读一遍，检查能否还原原数。

> **按需查阅 · 画完后核对表示方法**
> 从下标 0 开始依次存 0、7、0、5，有效长度是 4。两个 0 都有位置意义，不能丢掉。倒着读数组中的有效数位，才是 5070。

高精度加法，就是让数组做竖式

看懂一次完整相加：读两位数字和旧进位，留下当前位，送出新进位。

回到楼梯：第 10 阶有 89 种走法，第 11 阶有 144 种。第 12 阶需要完成的就是 **89 + 144**。我们用它学习原递推中的“一次加法”。

这一位的总和 s
= 第一个数的这一位 + 第二个数的这一位 + 传来的进位

留下个位

s % 10 是除以 10 的余数。比如 13 留下 3，写到结果的当前格。

送出进位

s / 10 是整数除法。比如 13 里面有 1 个完整的十，把 1 交给下一位。

同一个 s，要分别计算这两件事。不要先把 s 改成余数，否则会丢掉计算进位需要的信息。

89 + 144 的完整追踪

计算哪一位	这一位总和 s	留下 s % 10	送出 s / 10
个位 j = 0	9 + 4 + 0 = 13	3	1
十位 j = 1	8 + 4 + 1 = 13	3	1
百位 j = 2	0 + 1 + 1 = 2	2	0

结果数组从下标 0 开始是 3、3、2，按正常数位顺序输出就是 **233**。89 没有百位，计算时按 0 补齐；这就是竖式的“数位对齐”。

纸上演示：让 999 + 1 的进位走到最后

按“读两位与旧进位 → 计算 s → 留下一位 → 更新进位”的顺序填表。最后一行专门检查剩余进位。

处理数位	x 的数字	y 的数字	传入进位	总和 s	结果数字	送出进位
个位 j=0	9	1	0	____	____	____
十位 j=1	9	0	____	____	____	____
百位 j=2	9	0	____	____	____	____
最后检查	剩余进位：____；需要增加一位吗？____；按正常顺序输出：_____。					

为什么逐位加不会漏掉数值？
因为每次都有 s = 10 × 进位 + 当前位留下的数字。我们把完整的十交给高一位，把不足十的部分留在当前位。数值没有丢，只是分别放到了正确的位置。

按需查阅 · 大数已经很大了，s 还会溢出吗？

对于两个非负整数的十进制逐位相加，最大的单步总和是 9 + 9 + 1 = 19。因此每个数位、进位和 s 都可以用普通 int。整个数很长，不代表每一小步的数也很大。

04 / 完整规则

算对每一位，还要守住这四条规则

从“会算 89 + 144”，走到“换一组数也不会漏”。

① 位数不同：缺少的高位按 0 算

89 + 144 要处理到百位。先取两个有效长度的较大值 L，处理下标 0 到 L−1。本课约定输入数组中未使用的格子已经清零，并且容量足够。

② 最高位算完：再检查最后的进位

999 + 1 的百位算完后，还有一个 1 没有安家，要写入新的千位。结果可能比两个加数中较长的那个**多一位**。

若这时进位是 0，就不额外添一位。数组要为可能出现的新数位留出空间。

③ 开始新加法：状态从本次重新建立

每次相加前，进位都从 0 开始。结果数组先清零，有效长度重新计算。数组反复使用时尤其要防止旧数字混进新结果。

④ 输出：从最高有效位往个位读

若结果长为 lenZ，输出下标从 lenZ−1 到 0。不要把数组容量当成输出长度，也不能输出 z 就指望 C++ 自动打印所有数字。

0 也有一个数位。
结果为 0 时，要输出一个 0。不要因为寻找最高非零位而一路越界，也不要把结果输出成空白。

边界验证任务

用纸上追踪表计算 **999 + 1** 和 **0 + 0**。各用一句话说明：只按“较长加数的位数”计算并直接结束，会在哪一种情况下漏掉什么？你的输出规则又怎样照顾 0?

05 / 把原理翻译成代码

工具只有四件：数组、循环、取余、整除

先会做“一次大数相加”，再把它用于楼梯的多次递推。

工具	在这件事里做什么
数组	每格保存一个数位，多个格子组成一个数
循环	从个位开始，对每一位重复同样的计算
% 10	取出当前位应留下的数字
/ 10	算出要送给高一位的进位（整数除法）

先登记变量，每个名字只负责一件事

变量	含义	什么时候变化
x、y	两个加数的数位数组	一次加法中不修改它们
z	结果的数位数组	算完一位就写入一格

变量	含义	什么时候变化
lenX、lenY、lenZ	三个数各自的有效位数	读入或生成新数时确定
j	当前处理的数位下标	处理完一位后加 1
carry	尚未加入当前位的进位	每一轮算完后更新，传给下一轮
s	这一轮尚未拆分的总和	每轮重新计算

准备一次练习的数据

```
// 只用于练习 89 + 144；10 格是这个小练习的容量
int x[10] = {9, 8};
int y[10] = {4, 4, 1};
int z[10] = {};        // 所有元素初始化为 0
int lenX = 2, lenY = 3;
int carry = 0;
```

一轮计算的核心

```
int s = x[j] + y[j] + carry;
z[j] = s % 10;
carry = s / 10;
```

第一行读的是**上一位传来的旧进位**；第三行得到的是**送给下一位的新进位**。它们可以使用同一个变量，因为旧进位已经加进 s 了。

把三行放回一个完整过程

1. **准备：**确保输入数组空余位置为 0，结果清零，进位为 0。
2. **逐位：**确定较长加数的长度，用循环从个位到最高位执行这三行。
3. **收尾：**处理最后剩下的进位，确定结果的有效长度。
4. **输出：**从结果最高有效位向个位打印。

现在由你完成一个完整工具

用上面的变量和三行核心，自行补出循环、收尾和输出，写成计算 **89 + 144** 的小程序。完成后，改成 **999 + 1** 再试一次。重点不是照抄三行，而是让它们在正确的范围和顺序里工作。

> **按需查阅 · 如果不知道循环怎么组织**
>
> 先在纸上列出需要处理的下标，再决定起点与终点。两个加数的长度分别是 lenX 和 lenY，较大的那个告诉你要处理多少位。最后的进位检查放在这个循环之后。

> **按需查阅 · 如果不知道怎样保存最后一个进位**
>
> 假设刚处理了 L 位，那么这些下标是 0 到 L−1。新添的那一位应该在下标 L。根据最后的进位是否为 0，决定结果长度是 L 还是 L+1。

> **按需查阅 · 选读：如果加数很长，怎样从输入变成数组?**
>
> 用 string 读入十进制数字文本，再从右往左取字符。不要先读入 long long，否则拆分之前就已经放不下了。

```
string text;
cin >> text;
// 假设输入合法、无多余前导零，且数组容量足够
int lenX = text.size();
// 第 j 格来自文本从右往左的第 j 个字符：
x[j] = text[lenX – 1 – j] – '0';
```

'7' – '0' 得到整数 7。你需要自己写遍历各个字符的循环。本题“数楼梯”只输入较小的阶数 n，走法数是在计算中长大的，所以**楼梯输入不需要改成字符串**。

---

06 / 用回原题

一次加法会了，就接着算下一阶

沿用前面的 x、y、z 三个一维数组。这一节只新增一件事：算完一阶，更新前两项。

你已经会把 x 和 y 中的两个大数相加，结果存入 z。楼梯题反复需要的，正是这个加法工具。因为新一阶只用到前两阶，我们一直保留这两个走法数就够了。

1. 三个数组还是原来的工具

数组	准备计算第 i 阶时，保存什么	有效长度
x	前两阶，也就是第 i−2 阶的走法数	lenX
y	前一阶，也就是第 i−1 阶的走法数	lenY
z	本轮相加的结果：第 i 阶的走法数	lenZ

这里每个数组都表示**一个完整的大数**，每格仍只保存一位数字。前面学过的逐位相加、最后进位和有效长度规则，全部继续使用。

2. 算完一次，准备下一次

仍用前面那次 **89 + 144**。下面为方便阅读写出完整数值；数组里实际保存的仍是一个个数位。

时刻	x 保存	y 保存	z 保存
准备算第 12 阶	89（第 10 阶）	144（第 11 阶）	清零，等待结果
完成本次加法	89	144	233（第 12 阶）
更新后，准备算第 13 阶	144（第 11 阶）	233（第 12 阶）	下次相加前清零

相加的方法不变，变的是下一次交给它的两个数。
算完后，较旧的 89 已经用完；接下来需要的是 144 和 233。始终只保留眼下需要的几个数，这种方法叫“滚动保存”。

3. 更新时，先保护还要用的旧值

1. **先完成加法：**用 x、y 算出整个 z，包括最后进位和 lenZ。加法过程中不改 x、y。
2. **再复制旧 y 到 x：**把 y 的所有格子复制到 x，并让 lenX 变为 lenY。这样旧 y 已经保存好了。
3. **再复制 z 到 y：**把 z 的所有格子复制到 y，并让 lenY 变为 lenZ。
4. **下次相加前：**把 z 清零、carry 置 0，重新计算本次的 lenZ。

为什么不能反过来？如果先用 233 覆盖 y，再把 y 复制给 x，两个数组就都变成 233，原本还要用的 144 就丢了。

> **按需查阅 · 实现提示：怎样复制一个普通数组？**
>
> C++ 普通数组不能直接写 x = y。用一个循环，把 y 的各个格子逐一复制给 x；长度变量另外赋值。第一次实现可以复制整个数组容量，包含未使用位置的 0，避免留下旧数位。三个数组从一开始就都要初始化为 0。
>
> 复制只改变目标数组，不会把来源数组清空。必须等 z 已经复制给 y 后，才能清空 z。

4. 从第 3 阶开始重复这个过程

起步时，x 保存第 1 阶的走法数 1，y 保存第 2 阶的走法数 2，两个有效长度都是 1。接着从第 3 阶开始，每一阶都做一遍**“高精度相加 → 按顺序更新前两项”**，直到第 n 阶。

i 是外层循环中的楼梯阶数；j 是一次高精度加法里处理的数位下标。**外层每前进一阶，里面就完成一次整个大数的相加。**

按上面的约定，每轮相加后都更新 x、y，所以当 n≥2 时，最后的结果保存在 y 中，按 lenY 从高位向低位输出。n=2 不必进入循环；n=1 则取初始的 x。

> **按需查阅 · 实现准备：每个数组要多大？**
>
> 本题 x、y、z 各预留 **1100 格**即可，每格存一位，空余格保持为 0。数组长度看的是走法数有多少位，不是楼梯有多少阶。
>
> 容量依据（选读）：由递推可估计第 n 阶走法数不超过 1.62^n，而 1.62^5000 < 10^1048，1100 格留有余量。三个数组合计只需约 13 KB（按每个 int 为 4 字节计）。

完整任务：让同一个加法工具连续工作

从 x 表示 1、y 表示 2 开始，在纸上连续推到第 5 阶。每轮记录“相加前的 x、y → 得到的 z → 更新后的 x、y”，同时记下三个有效长度。

能解释每次更新为什么没有丢掉下一轮要用的数，再把这个过程接进自己的楼梯程序。不要只验证某一次加法，要验证它能连续工作。

---

07 / 独立应用与验证

先把工具做稳，再用它解决题目

每次练习都有完整产出。先自己尝试，确实卡住时再看提示。

任务 A · 完成一次高精度加法

写出能处理两个非负整数相加的小程序。第一版可以直接给数组赋值，先不处理字符串输入。自己负责循环范围、最后进位、有效长度和输出。

用什么测试	检查什么
89 + 144	位数不同、进位、原题中的一次递推
999 + 1	连续进位、结果多一位
0 + 0	最小值也能正确输出
5070 + 30	中间和末尾的零没有被丢掉

自测 1：你的程序算出 999 + 1 是多少？　运行结果：_____

自测 2：你的程序算出 5070 + 30 是多少？　运行结果：_____

任务 B · 把工具接回数楼梯

保留原来的递推思路，升级数据表示、相加过程和输出。由你完成代码；这份课件不给整题的可提交程序。

测试输入	验证重点
n = 1、2	不进入递推时，初始化和输出是否仍正确
n = 4、12	小范围手推与程序一致，数位方向正确
n = 91、92、93	跨过旧 long long 的范围后仍能精确递推
n = 5000	容量、边界、运行时间和内存能否承受

“变大了”“没崩溃”不能证明大数正确。先核对小结果，再用独立的逐位加法检查跨界测试中的相邻三项关系；最终结合评测结果验证。不要继续拿已溢出的旧程序当标准答案。

任务 C · 迁移到另一道题

两家图书馆分别报告了一个很长的藏书册数，要求精确求总数。请说明：能复用高精度加法的哪部分？这里还需要“楼梯的外层循环”吗？

> **按需查阅 · 做完以后检查迁移思路**
> 可以复用数字表示、逐位加法、最后进位和输出。两个很长的输入要从字符串转换成数位数组。只相加一次，不需要楼梯递推；工具相同，组织工具的题目主线不同。

什么时候想到这个工具?
题目要求精确整数结果，结果可能超过普通整数范围，而且核心运算是加法时，可以考虑高精度加法。如果题目明确只要求对某个数取模，就按那道题的输出要求建模，不必总是保存整个大数。

---

08 / 留给自己的方法

把方法带走，不只记住三行代码

一张方法卡
为什么：结果超过普通整数范围，但必须精确。
怎样存：一格一位，个位在下标 0，另记有效长度。
怎样算：从低位到高位，加上旧进位，留下余数，送出新进位。
怎样收尾：补最后进位，确定长度，从高位向低位输出。
怎样用：题目原来的数量关系不变，用这个工具完成其中的加法。

交给评测前，我能独立完成这些事吗?

☐ 我能说明数组容量、有效长度和下标分别是什么。

☐ 我能追踪一次有连续进位的加法，解释每个变量的作用。

☐ 我能独立写出循环、收尾和输出，并通过三类测试。

☐ 我能按顺序更新 x、y，并说清楼梯阶数 i 与数位下标 j 的区别。

这些是你的自查记录，不是自动认定“已经掌握”。隔天不看课件再做一次，更能看出哪些地方真正会了。

我的设计与还不确定的地方

改对后，我自己的反思